

Introduction to Databases

- The database serves as the backbone of the Smart Healthcare Management Platform. It ensures the preservation, consistency, and integrity of all medical, financial, and administrative data, while providing a real-time, secure connection between various hospital departments (doctors, patients, laboratories, radiology, pharmacy, and the reception desk). The data architecture is designed to achieve maximum query efficiency and to guarantee system stability under the heavy data loads generated by daily transactions within the medical facility.

2. DBMS Selection & Development Environment

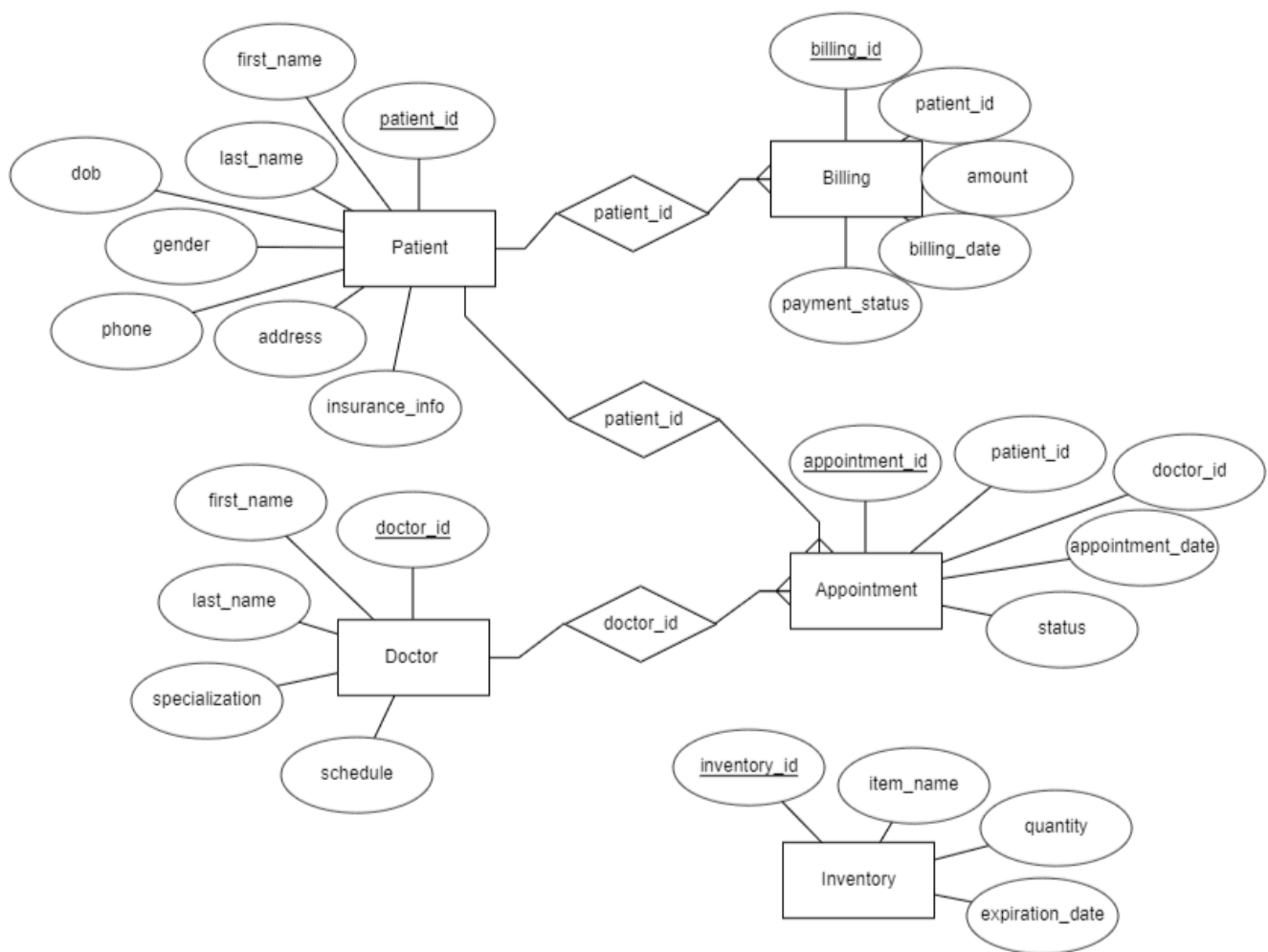
The reliable MySQL Relational Database Management System (RDBMS) was selected to build the platform's database. This engineering choice is backed by several technical reasons:

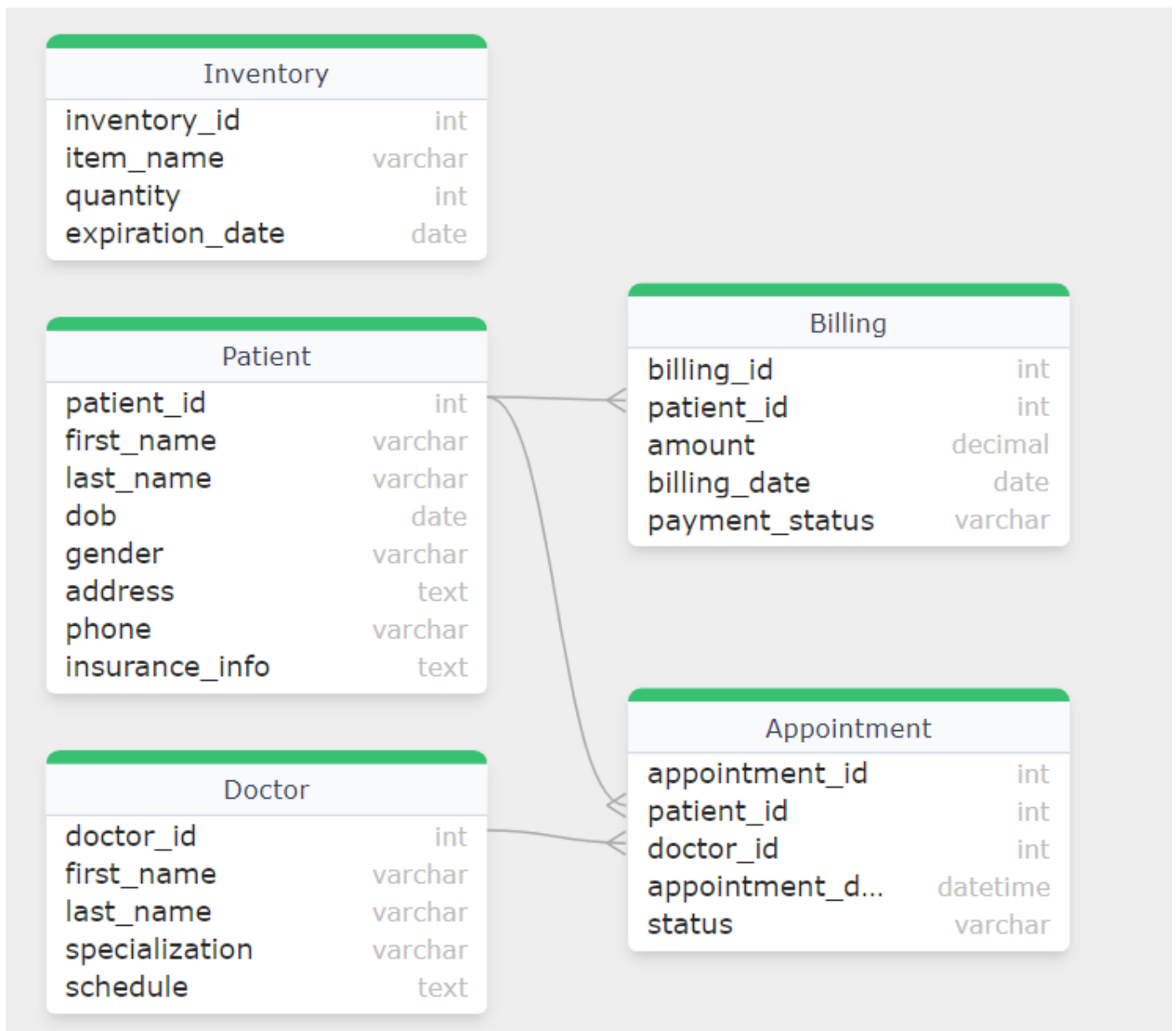
- **Support for Complex Transactions (ACID Properties):** MySQL ensures data integrity and consistency during concurrent modification processes, such as managing simultaneous appointment bookings to prevent double-booking or recording precise financial payments.
- **High Compatibility:** MySQL integrates seamlessly with backend development frameworks and programming languages (whether utilizing PHP, Python, or Node.js).
- **Speed and Performance:** It offers high capability in processing complex queries and efficiently executing JOIN operations across large-scale tables.

Part 1: Structure of the Appointments Table (appointments)

The table is designed to cover all booking details within the hospital system and contains the following fields:

- **Prescriptions Table:** Represents the interactive medical hub that links the physician directly to the pharmacy, laboratory, and radiology departments.
- **Lab Results Table:** Stores quantitative data and uploads technical laboratory reports.
- **Radiology Results Table:** Manages diagnostic imaging summaries and links them directly to the patient's record.
- **Payments Table:** Documents and tracks financial transactions and invoice statuses.





4. Data Dictionary & Detailed Table Structures

Below is the precise technical breakdown for each table, defining fields, data types, and integrity constraints such as Primary Keys (PK) and Foreign Keys (FK):

1. Patients Table

This table handles the storage of patients' personal and medical demographics. The system utilizes **Automated ID Generation** to assign a unique identifier to each patient upon initial registration.

- **Patient_ID** (*int, Primary Key, Auto Increment*): The unique identifier for the patient.
- **Name** (*varchar(150), Not Null*): The full name of the patient.
- **Age** (*int*): The age of the patient.
- **Phone** (*varchar(20)*): The contact number used for communication and verification.

- Email (*varchar(100), Unique*): The patient's email address (integrated with the Gmail API for automated delivery of medical reports).
- Medical_History (*text*): Permanent medical notes, chronic illnesses, and historical health records.

2. Doctors Table

- Doctor_ID (*int, Primary Key, Auto Increment*): The unique identifier for the physician.
- Name (*varchar(150), Not Null*): The full name of the doctor.
- Specialization (*varchar(100)*): The medical specialty (e.g., Internal Medicine, Pediatrics, Surgery, Cardiology).
- Phone (*varchar(20)*): The doctor's contact number.
- Availability_Status (*boolean*): A flag indicating whether the doctor is currently available to accept new bookings.

```
CREATE TABLE `users` (
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email_verified_at` timestamp NULL DEFAULT NULL,
  `password` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `remember_token` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

3. Appointments Table

This table manages patient flow, optimizes waiting times, and dynamically prevents booking conflicts.

- **1. Design Philosophy**

The appointments table within the schema is not merely a passive record-keeper; it serves as the dynamic engine driving **Patient Flow Management** within the hospital. It is architected as a centralized relational hub that seamlessly bridges four core hospital domains: the Patient, the Doctor, the Department, and the Financial registry.

- **2. Architectural Columns Analysis**

The table columns can be logically classified into four functional categories based on their operational roles:

- **A. Relational Keys & Links**

id (bigint UNSIGNED): The auto-incrementing Primary Key. Utilizing bigint ensures the system can accommodate millions of appointments over years of hospital operations without running out of numerical bounds.

patient_id & doctor_id: These serve as Foreign Keys linking each appointment directly to the respective patient and doctor profiles, ensuring strict **Referential Integrity** across the database.

department_id: A strategic foreign key that routes the appointment to its specific department (e.g., Outpatient Clinics, Labs, Radiology), preventing scheduling silos.

- **B. Scheduling & Timing Constraints**

- **appointment_date (date):** Records the calendar date allocated for the appointment.

- **start_time & end_time (time):** This highlights the system's advanced scheduling logic. Instead of relying on vague time frames, it defines a precise time slot (e.g., 10:00 AM to 10:30 AM), effectively eliminating **Slot Overlapping** and scheduling conflicts.

- **queue_number (int):** Represents the sequential live queue position of the patient on that specific day, which forms the basis for the hospital's waiting room management.

- **C. Operational Workflow & Status Trackers**

-**status (varchar):** Manages the entire **Appointment Lifecycle**. Every entry initiates as 'pending' and transitions through system-defined operational states (e.g., 'confirmed', 'cancelled', 'completed').

-**checked_in_at (timestamp):** Captures the exact moment the patient physically arrives at the reception desk to check in. By querying the delta between checked_in_at and start_time, the system can compute critical key performance indicators (KPIs) like **Patient Lateness Rates** and **Average Wait Times**.

-**auto_confirmed_at:** Dedicated to automated system triggers that confirm bookings instantly without requiring manual human intervention once specific criteria are met.

- **D. Medical & Financial Records**

-**treatment:** Explicitly logs the clinical purpose of the visit (e.g., "Cardiac Consultation").

-**notes (text)**: A flexible, long-form text field designed to capture initial patient complaints or clinical remarks. It fully supports Arabic characters due to the global utf8mb4 character encoding applied in the database dump.

-**fee (decimal(10,2))**: A high-precision financial column tracking the cost of the appointment. Choosing decimal instead of float or double is an industry standard that guarantees zero rounding errors during financial reporting.

- **3. The Appointment Data Lifecycle**

-Based on the transactional data within the SQL dump, the system processes each appointment through a structured pipeline:

-**Creation Stage**: The record is initialized with automated created_at timestamps, defaulting the status to 'pending'.

-**Arrival Stage (Check-in)**: Upon physical arrival, the receptionist updates the record, triggering the checked_in_at timestamp and assigning a sequential queue_number.

-**Closure Stage**: The transactional fee is finalized, and the status shifts to its final operational state (e.g., 'completed').

```
CREATE TABLE `appointments` (  
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,  
  `patient_id` bigint UNSIGNED NOT NULL,  
  `doctor_id` bigint UNSIGNED NOT NULL,  
  `department_id` bigint UNSIGNED DEFAULT NULL,  
  `appointment_date` date NOT NULL,  
  `start_time` time NOT NULL,  
  `end_time` time NOT NULL,  
  `status` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'pending',  
  `queue_number` int UNSIGNED DEFAULT NULL,  
  `auto_confirmed_at` timestamp NULL DEFAULT NULL,  
  `checked_in_at` timestamp NULL DEFAULT NULL,  
  `confirmation_email_sent_at` timestamp NULL DEFAULT NULL,  
  `treatment` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `notes` text COLLATE utf8mb4_unicode_ci,  
  `phone_number` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `fee` decimal(10,2) NOT NULL DEFAULT '0.00',  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4. Prescriptions Table

This table acts as the cornerstone of the "Electronic Prescription System." Once saved by a doctor, the record instantly routes data to the pharmacy and diagnostic labs.

- **Prescription_ID** (*int, Primary Key, Auto Increment*): The unique prescription ID.
- **Doctor_ID** (*int, Foreign Key references Doctors*): The prescribing physician.
- **Patient_ID** (*int, Foreign Key references Patients*): The target patient.
- **Diagnosis_Notes** (*text*): Clinical diagnosis notes recorded during the examination.
- **Medication** (*text*): Detailed prescribed drugs, dosages, and administration schedules.
- **Lab_Required** (*text*): Requested laboratory tests (if applicable).
- **Radiology_Required** (*text*): Requested diagnostic imaging or scans (if applicable).
- **Issue_Date** (*timestamp*): The exact timestamp of creation.

```
CREATE TABLE `diagnoses` (  
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,  
  `appointment_id` bigint UNSIGNED NOT NULL,  
  `patient_id` bigint UNSIGNED NOT NULL,  
  `doctor_id` bigint UNSIGNED NOT NULL,  
  `symptoms` text COLLATE utf8mb4_unicode_ci NOT NULL,  
  `diagnosis` text COLLATE utf8mb4_unicode_ci NOT NULL,  
  `prescription` text COLLATE utf8mb4_unicode_ci,  
  `treatment_plan` text COLLATE utf8mb4_unicode_ci,  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  KEY `diagnoses_appointment_id_foreign` (`appointment_id`),  
  KEY `diagnoses_patient_id_foreign` (`patient_id`),  
  KEY `diagnoses_doctor_id_foreign` (`doctor_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

5. Lab Results Table

- **Result_ID** (*int, Primary Key, Auto Increment*): The unique test result identifier.
- **Patient_ID** (*int, Foreign Key references Patients*): The patient tested.
- **Prescription_ID** (*int, Foreign Key references Prescriptions*): The reference prescription that ordered the lab work.

- Test_Name (*varchar(100)*): The medical name of the panel or test (e.g., CBC, Liver Function Tests).
- Result_Value (*text*): The lab values, findings, or textual diagnostic report.
- Uploaded_File_Path (*varchar(255)*): The directory path of the uploaded report file (PDF) for direct cloud access and viewing.
- Upload_Date (*timestamp*): The timestamp of when the technician uploaded the results.

6. Radiology Results Table

- Radiology_ID (*int, Primary Key, Auto Increment*): The unique radiology report ID.
- Patient_ID (*int, Foreign Key references Patients*): The patient examined.
- Test_Name (*varchar(100)*): The imaging modality used (e.g., X-Ray, MRI, CT Scan).
- Report_Details (*text*): The professional diagnostic narrative written by the radiologist or technician.
- File_Path (*varchar(255)*): The digital storage path for the scan image or digital report.
- Date (*timestamp*): The exact time of the radiological procedure.

```
CREATE TABLE `scan_results` (
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,
  `scan_request_id` bigint UNSIGNED NOT NULL,
  `patient_id` bigint UNSIGNED NOT NULL,
  `doctor_id` bigint UNSIGNED DEFAULT NULL,
  `entered_by_id` bigint UNSIGNED DEFAULT NULL,
  `findings` text COLLATE utf8mb4_unicode_ci NOT NULL,
  `impression` text COLLATE utf8mb4_unicode_ci,
  `scan_image` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `scan_results_scan_request_id_foreign` (`scan_request_id`),
  KEY `scan_results_patient_id_foreign` (`patient_id`),
  KEY `scan_results_doctor_id_foreign` (`doctor_id`),
  KEY `scan_results_entered_by_id_foreign` (`entered_by_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

7. Payments Table

Designed to eliminate processing delays and revenue tracking errors common in traditional, manual hospital billing systems.

- **Payment_ID** (*int, Primary Key, Auto Increment*): The unique financial transaction ID.
- **Patient_ID** (*int, Foreign Key references Patients*): The patient responsible for payment settlement.
- **Amount** (*decimal(10,2)*): The precise monetary amount owed/paid.
- **Payment_Purpose** (*varchar(150)*): The reason for the invoice (e.g., Consultation Fee, Lab Test, Scan).
- **Status** (*enum: 'Paid', 'Pending', 'Refunded'*): The financial lifecycle status of the transaction.
- **Transaction_Date** (*timestamp*): The timestamp when the payment transaction was officially processed.

5. Relationships & Integrity Constraints

To eliminate structural anomalies and prevent the loss of critical medical records (**Data Integrity**), the following engineering rules have been enforced at the schema level:

- **One-to-Many (1:N) Relationship between Patients and Operational Data (Appointments / Prescriptions / Payments)**: A single patient (**Patient_ID**) can book multiple appointments, own an extensive history of medical prescriptions, and execute various financial transactions over time. However, any individual booking, prescription, or invoice must belong strictly to one specific patient.
- **One-to-Many (1:N) Relationship between Doctors and Operational Data (Appointments / Prescriptions)**: An individual physician can manage an array of scheduled appointments, consult multiple patients, and issue numerous independent electronic prescriptions.
- **Referential Action Control (ON UPDATE CASCADE / ON DELETE RESTRICT)**: The system completely restricts (**RESTRICT**) the deletion of any doctor or patient record if it is linked to existing medical or financial rows. This strictly complies with healthcare regulations demanding the indefinite preservation of patient health histories. Conversely, if a primary user ID is modified, the changes

automatically propagate (CASCADE) through all relational tables to maintain structural unity.

```
-- 1. Constraints for table: scan_requests
ALTER TABLE `scan_requests`
  ADD CONSTRAINT `scan_requests_diag_fk`
    FOREIGN KEY (`diagnosis_id`)
    REFERENCES `diagnoses` (`id`)
    ON DELETE CASCADE,

  ADD CONSTRAINT `scan_requests_doc_fk`
    FOREIGN KEY (`doctor_id`)
    REFERENCES `doctors` (`id`)
    ON DELETE SET NULL,

  ADD CONSTRAINT `scan_requests_pat_fk`
    FOREIGN KEY (`patient_id`)
    REFERENCES `patients` (`id`)
    ON DELETE CASCADE;

-- 2. Constraints for table: scan_results
ALTER TABLE `scan_results`
  ADD CONSTRAINT `scan_results_req_fk`
    FOREIGN KEY (`scan_request_id`)
    REFERENCES `scan_requests` (`id`)
    ON DELETE CASCADE,

  ADD CONSTRAINT `scan_results_pat_fk`
    FOREIGN KEY (`patient_id`)
    REFERENCES `patients` (`id`)
    ON DELETE CASCADE,

  ADD CONSTRAINT `scan_results_doc_fk`
    FOREIGN KEY (`doctor_id`)
    REFERENCES `doctors` (`id`)
    ON DELETE SET NULL,

  ADD CONSTRAINT `scan_results_user_fk`
    FOREIGN KEY (`entered_by_id`)
    REFERENCES `users` (`id`)
    ON DELETE SET NULL;
```

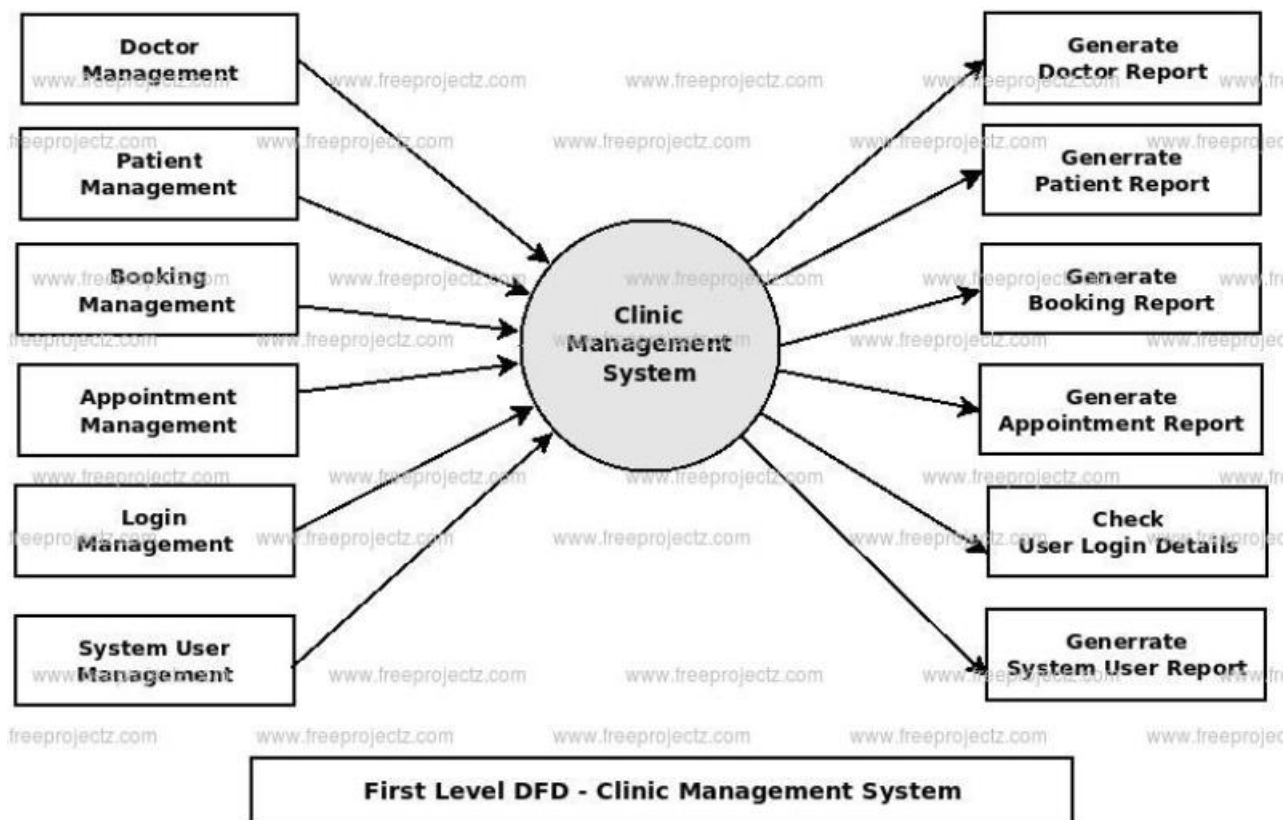
6. Data Workflow Execution

Data moves dynamically through the relational schema following the logical sequence of a patient's journey within the smart facility:

1. **The Booking Stage:** The patient sets up an account. The Patients table captures the details and automatically triggers an ID

generation. Simultaneously, a corresponding record is inserted into the Appointments table.

2. **The Examination Stage:** The physician queries the patient's record from the Patients table via their unique ID. Following the check-up, the clinical diagnosis, medication list, and diagnostic orders are stored within the Prescriptions table.
3. **The Integration and Distribution Stage:** Immediately upon the insertion of a prescription record, the pharmacy, lab, and radiology interfaces actively listen via targeted SQL queries to read outstanding medication or testing orders in real time.
4. **The Financial and Medical Update Stage:** The laboratory updates the system by writing data to the Lab Results table, immediately making it visible on the patient's portal, while the reception desk updates the status of the Payments table upon invoice settlement.

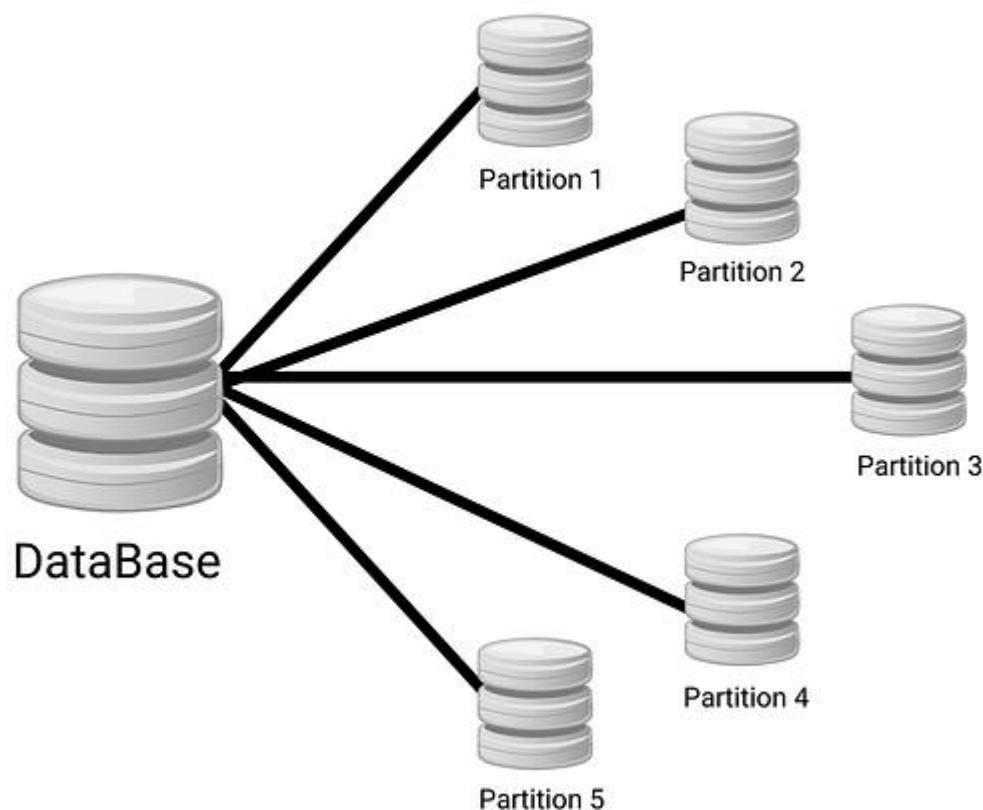


5. Future Database Enhancements (التطوير المستقبلي لقاعدة البيانات)

To ensure the scalability, high availability, and long-term efficiency of the hospital management system as it transitions from a localized prototype to an enterprise-grade medical solution, several advanced database methodologies will be integrated in future phases:

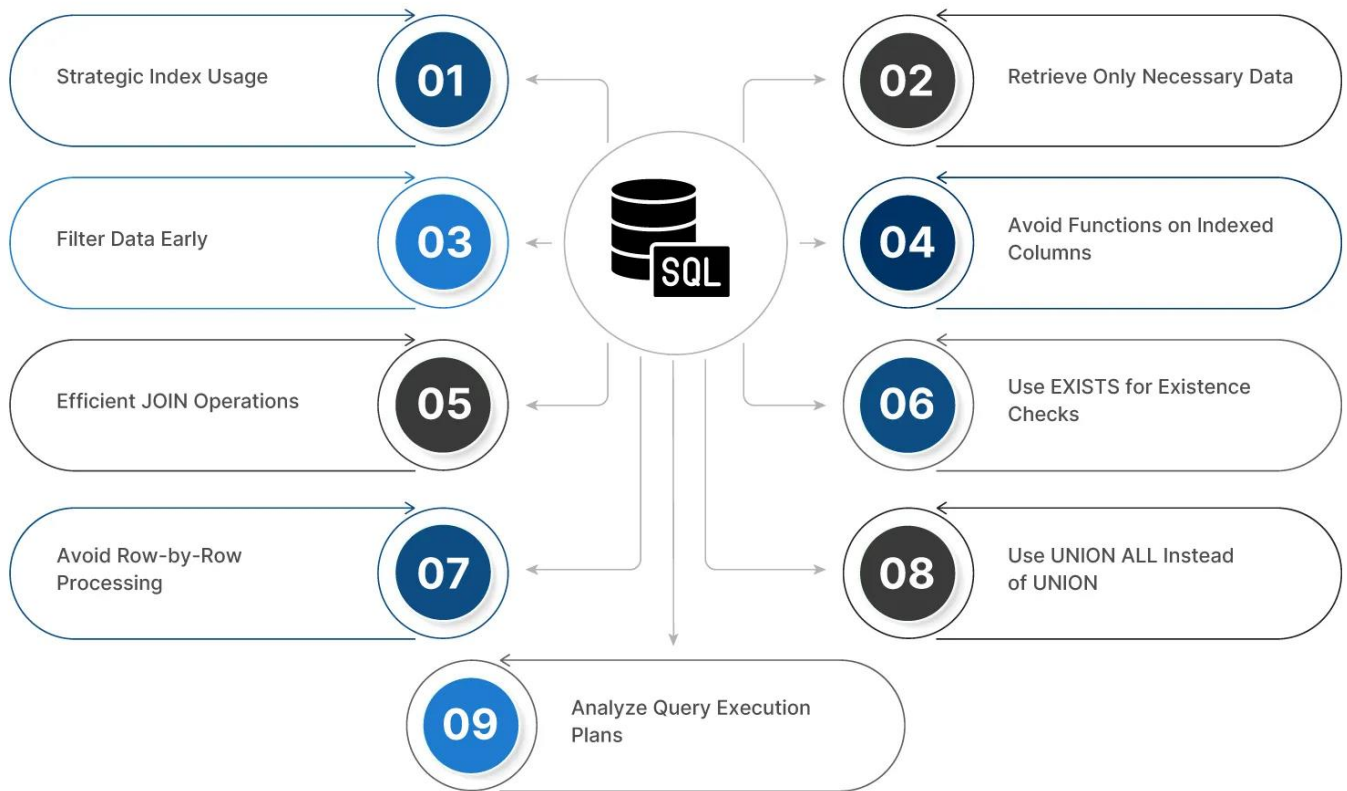
A. Database Partitioning & Sharding

As the hospital expands and the appointments table accumulates millions of transactional records over time, performance bottlenecks could arise during standard queries. To mitigate this, **Horizontal Partitioning (Sharding)** will be implemented. For instance, the appointments table can be partitioned by `appointment_date` (e.g., creating separate physical partitions for each calendar year). This dramatically reduces query execution time and optimizes index lookups.



B. Index Optimization for High-Frequency Queries

To accommodate rapid real-time lookups at the hospital reception desk, custom composite indexes will be deployed. Creating indexes on combined columns such as `(doctor_id, appointment_date)` or `(patient_id, status)` will ensure that the system fetches operational data with sub-millisecond latency, even during peak hospital hours.



C. Advanced Archiving Strategy & Data Warehousing

To maintain a lean and fast production database, an automated archiving pipeline will be developed. Historical data (e.g., completed or cancelled appointments older than three years) will be automatically migrated to a dedicated **Data Warehouse (OLAP system)**. This preserves historical medical records for long-term analytics and reporting without slowing down day-to-day hospital operations.

D. Multi-Tenant Architecture & Data Isolation

If the application expands to serve a network of multiple hospital branches or clinics, the database schema will be upgraded to a **Multi-Tenant Architecture**. This will be achieved either through a "Shared Database, Separate Schemas" approach or by introducing a strict `branch_id` tenant identifier across all operational tables, enforcing absolute data isolation and security between different healthcare facilities.

E. Replication and High Availability (HA)

To achieve zero-downtime and robust disaster recovery, a **Master-Slave Replication** topology will be configured. The Master database will handle all write operations (such as creating new appointments or processing payments), while one or more Slave databases will handle read-heavy operations (like generating management reports or viewing patient histories), ensuring continuous system availability even during hardware failures.

3. Entity-Relationship Diagram (ERD)

3.1 ERD Overview

The following Entity-Relationship Diagram illustrates the complete relational architecture of the Smart Healthcare Management Platform database. It maps all core entities, their attributes, primary and foreign key linkages, and the cardinality of every relationship. This diagram serves as the authoritative blueprint for understanding how data flows between departments, patients, clinicians, and financial records within the system.

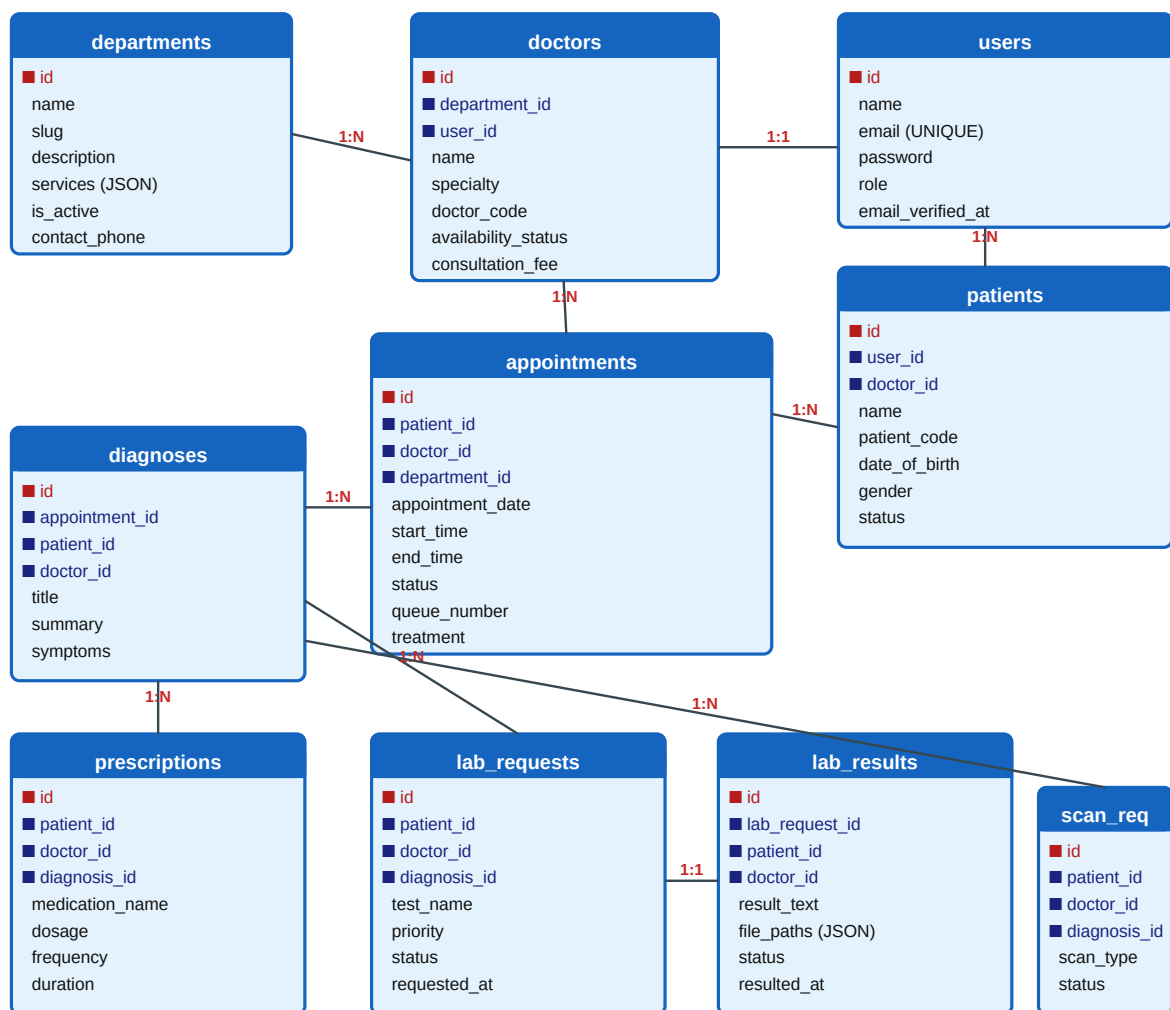


Figure 1 — Complete ERD of the Smart Healthcare Management Platform Database

3.2 ERD Relationship Descriptions

The table below enumerates every relationship encoded in the ERD, specifying the participating entities, cardinality, and enforcing foreign key constraint.

Parent	Child	Cardinality	FK Column	ON DELETE
departments	doctors	1 : N	doctors.department_id	CASCADE
users	doctors	1 : 1 (opt)	doctors.user_id	SET NULL
users	patients	1 : 1 (opt)	patients.user_id	SET NULL
doctors	patients	1 : N	patients.doctor_id	SET NULL
patients	appointments	1 : N	appointments.patient_id	RESTRICT
doctors	appointments	1 : N	appointments.doctor_id	RESTRICT
departments	appointments	1 : N	appointments.department_id	SET NULL
appointments	diagnoses	1 : 1 (opt)	diagnoses.appointment_id	SET NULL
patients	diagnoses	1 : N	diagnoses.patient_id	CASCADE
doctors	diagnoses	1 : N	diagnoses.doctor_id	SET NULL
diagnoses	prescriptions	1 : N	prescriptions.diagnosis_id	CASCADE
patients	prescriptions	1 : N	prescriptions.patient_id	CASCADE
doctors	prescriptions	1 : N	prescriptions.doctor_id	SET NULL
diagnoses	lab_requests	1 : N	lab_requests.diagnosis_id	CASCADE
patients	lab_requests	1 : N	lab_requests.patient_id	CASCADE
lab_requests	lab_results	1 : 1	lab_results.lab_request_id	CASCADE
diagnoses	scan_requests	1 : N	scan_requests.diagnosis_id	CASCADE
scan_requests	scan_results	1 : 1	scan_results.scan_request_id	CASCADE

3.3 Key Relationship Notes

- **departments → doctors (1:N, CASCADE):** Each department contains one or more doctors. Deleting a department cascades to remove all its affiliated doctors, suitable for decommissioning entire clinical units.
- **patients ↔ appointments (1:N, RESTRICT):** A patient may have many appointments over their lifetime, but each appointment belongs to exactly one patient. RESTRICT prevents deleting a patient who still has outstanding records, safeguarding medical history.
- **appointments → diagnoses (1:1 optional):** An appointment may produce at most one diagnosis record. The optional relationship reflects that cancelled or unfulfilled appointments generate no diagnosis.
- **diagnoses → prescriptions / lab_requests / scan_requests (1:N):** A single consultation can produce multiple prescriptions or diagnostic orders, supporting

multi-drug regimens and parallel diagnostic workflows in one visit.

- **lab_requests** → **lab_results (1:1, CASCADE)**: Each lab order yields exactly one result record. CASCADE ensures results are automatically removed when their originating request is deleted.

7. Database Normalization

Database normalization is the systematic process of structuring a relational database to minimize data redundancy and prevent update, insertion, and deletion anomalies. The Smart Healthcare Management Platform database has been architected to comply with the first three Normal Forms (1NF, 2NF, 3NF) and the Boyce-Codd Normal Form (BCNF). Each form is explained and demonstrated against the actual schema.

7.1 First Normal Form (1NF)

Definition. A table is in 1NF if: (a) all column values are atomic; (b) there are no repeating groups; and (c) every row is uniquely identifiable via a primary key.

Compliance evidence:

- **Atomic values:** Every column stores a single indivisible value. The `patients` table, for instance, separates `name`, `date_of_birth`, and `gender` into distinct columns rather than concatenating them.
- **Unique primary keys:** Every table defines `id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT` as its primary key, guaranteeing row-level uniqueness.
- **No repeating groups:** Multi-valued data (e.g., a department's service offerings) is stored as a typed `JSON` column — MySQL's native, queryable format. A strictly relational 1NF interpretation could extract this into a `department_services` junction table in a future enhancement.
- **NOT NULL enforcement:** Logically mandatory columns such as `treatment` in `appointments` and `name` in `patients` are declared `NOT NULL`, enforcing data completeness at the schema level.

7.2 Second Normal Form (2NF)

Definition. A table is in 2NF if it is in 1NF and every non-key attribute is *fully functionally dependent* on the *entire* primary key — not a subset of it. 2NF is only relevant when the primary key is composite.

Compliance evidence:

- **Single-column surrogate keys:** Every table uses a single `AUTO_INCREMENT id` column as its primary key. Because no composite key exists, partial dependencies are structurally impossible and 2NF is trivially satisfied.
- **No partial dependencies:** In the `appointments` table, all attributes (`fee`, `status`, `treatment`, `queue_number`) are fully determined by the appointment's `id` — not by any sub-portion of a composite key.
- **Lookup data separated:** Doctor specialisation and department names are stored in their own tables and referenced via foreign keys — not duplicated inside the `appointments` table, further enforcing full dependency.

7.3 Third Normal Form (3NF)

Definition. A table is in 3NF if it is in 2NF and no non-key attribute is *transitively dependent* on the primary key — i.e., no non-key column determines another non-key column.

Compliance evidence:

- **Doctor data isolated:** The appointments table stores only doctor_id. Doctor-specific data (name, specialty, consultation_fee) lives solely in doctors, removing the transitive chain: appointment_id → doctor_id → doctor_name.
- **Department data isolated:** Department names and contact details reside exclusively in departments. Appointments hold only department_id, eliminating: appointment_id → department_id → department_name.
- **Patient demographics isolated:** Patient addresses, dates of birth, and notes are stored in patients and referenced by patient_id, preventing transitive dependencies across the prescriptions or lab tables.
- **Financial precision:** The fee DECIMAL(10, 2) in appointments is directly determined by the appointment's PK and is not derivable from any other non-key column in the same row.

7.4 Boyce-Codd Normal Form (BCNF)

Definition. A relation is in BCNF if, for every non-trivial functional dependency X → Y, X is a superkey of the relation (i.e., X uniquely identifies every row).

All tables in this schema rely on a single surrogate primary key. Because no non-key attribute determines another non-key attribute, every functional dependency takes the form id → attribute, and id is always a superkey — confirming BCNF compliance. The users table additionally carries a UNIQUE constraint on email, making email a candidate key. All other attributes depend on either id or email — both superkeys — preserving BCNF.

7.5 Normalization Compliance Summary

Normal Form	Core Requirement	Status	Key Evidence in Schema
1NF	Atomic values, unique PKs, no repeating groups	✓ Met	Single-column PKs; NOT NULL on mandatory fields; JSON for multi-value data
2NF	No partial dependencies on composite key	✓ Met	All PKs are single-column; partial dependency structurally impossible
3NF	No transitive dependencies	✓ Met	Doctor, patient & department data fully normalised into own tables
BCNF	Every functional determinant is a superkey	✓ Met	Surrogate PKs + unique email constraint; no anomalous FDs present

8. Departments Table

8.1 Purpose

The departments table is the administrative apex of the hospital's clinical hierarchy. It defines every clinical unit within the Smart Healthcare Management Platform — from Cardiology and Oncology to Vascular Surgery and Internal Medicine. As the parent entity in the system, it directly governs which doctors may be assigned to a specialty and which appointment types are valid for each unit.

The table serves a dual role: it drives the operational relational logic by providing the `department_id` foreign key referenced across the doctors and appointments tables, and it stores patient-facing content (descriptions, service lists, hero images, and contact details) that powers the public hospital portal — making it the single source of truth for both the backend and the UI.

8.2 Schema Definition

```
CREATE TABLE `departments` (  
  `id` bigint UNSIGNED NOT NULL AUTO_INCREMENT,  
  `name` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `slug` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `description` text COLLATE utf8mb4_unicode_ci,  
  `services` json DEFAULT NULL,  
  `icon` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `hero_image` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `sidebar_image` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `contact_phone` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `contact_email` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
  `is_active` tinyint(1) NOT NULL DEFAULT '1',  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `departments_slug_unique` (`slug`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

8.3 Column Definitions

Column	Data Type	Constraint	Description
id	BIGINT UNSIGNED	PK, AI	Unique auto-incrementing department identifier
name	VARCHAR(255)	NOT NULL	Full display name, e.g., "Cardiology"
slug	VARCHAR(255)	NOT NULL, UNIQUE	URL-safe identifier, e.g., "cardiology"
description	TEXT	NULLABLE	Patient-facing clinical overview of the department
services	JSON	NULLABLE	Ordered list of clinical procedures offered
icon	VARCHAR(255)	NULLABLE	File path to the department icon image
hero_image	VARCHAR(255)	NULLABLE	Path to the banner/hero image
sidebar_image	VARCHAR(255)	NULLABLE	Path to the sidebar portrait image

contact_phone	VARCHAR(255)	NULLABLE	Department direct telephone line
contact_email	VARCHAR(255)	NULLABLE	Department contact email address
is_active	TINYINT(1)	DEFAULT 1	Soft-delete: 1 = active, 0 = hidden from portal
created_at	TIMESTAMP	NULLABLE	Row creation timestamp (auto-managed by Laravel)
updated_at	TIMESTAMP	NULLABLE	Last modification timestamp (auto-managed)

8.4 Indexes

Index	Type	Column(s)	Purpose
PRIMARY	Primary Key	id	Unique row identification; used in all FK joins from doctors and appointments
departments_slug_unique	UNIQUE	slug	Ensures URL slug uniqueness; enables O(log n) slug-based portal lookups
[Recommended] idx_dept_active	INDEX	is_active	Speeds up filtering of active departments in portal queries

8.5 Sample Seed Data

The following records represent the 13 active departments loaded into the system during the initial database seeding phase. The full dataset contains 13 departments spanning major clinical specialties:

id	name	slug	contact_phone	is_active
2	Cardiology	cardiology	+20 2 1234 5678	1
14	Dentistry	dentistry	+20 2 1234 5678	1
15	ENT	ent	+20 2 1234 5678	1
16	General Surgery	general-surgery	+20 2 1234 5678	1
17	Neurosurgery	neurosurgery	+20 2 1234 5678	1
18	Obs & Gyn	obs-gyn	+20 2 1234 5678	1
19	Oncology	oncology	+20 2 1234 5678	1
20	Orthopedic	orthopedic	+20 2 1234 5678	1
22	GIT	git	+20 2 1234 5678	1
23	Internal Medicine	internal-medicine	+20 2 1234 5678	1
26	Radiodiagnosis	radiodiagnosis	+20 2 1234 5678	1
27	Cardiothoracic	cardiothoracic	+20 2 1234 5678	1
28	Vascular Surgery	vascular-surgery	+20 2 1234 5678	1

8.6 Relationship Explanation

- **departments → doctors (1:N, ON DELETE CASCADE):** A department contains one or more doctors. The CASCADE rule means that retiring a department automatically removes all its affiliated physician records, preventing orphaned rows.
- **departments → appointments (1:N, ON DELETE SET NULL):** Appointments reference a department via department_id. The SET NULL action preserves historical appointment records even if a department is decommissioned, setting the

department reference to NULL to maintain audit integrity.

9. Laravel Integration

The Smart Healthcare Management Platform backend is built with **Laravel 11** on **PHP 8.2+**, connected to the MySQL 8.4 database through the Eloquent ORM. This section provides a concise technical reference covering migration definitions, Eloquent model declarations, relationship methods, controller patterns, route definitions, and API response structures.

Environment: Laravel 11 | PHP 8.2+ | MySQL 8.4 | Laravel Sanctum (API authentication)

9.1 Migration Example — departments

Migrations in Laravel define the database schema in version-controlled PHP files, enabling repeatable, team-shared schema management. The migration below creates the departments table:

```
<?php
// database/migrations/xxxx_create_departments_table.php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('departments', function (Blueprint $table) {
            $table->id(); // BIGINT UNSIGNED PK AI
            $table->string('name');
            $table->string('slug')->unique(); // URL slug – UNIQUE index
            $table->text('description')->nullable();
            $table->json('services')->nullable(); // Array of clinical services
            $table->string('icon')->nullable();
            $table->string('hero_image')->nullable();
            $table->string('sidebar_image')->nullable();
            $table->string('contact_phone')->nullable();
            $table->string('contact_email')->nullable();
            $table->boolean('is_active')->default(true);
            $table->timestamps(); // created_at & updated_at
        });
    }
    public function down(): void { Schema::dropIfExists('departments'); }
};
```

9.2 Eloquent Model Examples

Department Model

```

<?php // app/Models/Department.php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Department extends Model {
protected $fillable = [
    'name', 'slug', 'description', 'services', 'icon',
    'hero_image', 'sidebar_image', 'contact_phone',
    'contact_email', 'is_active',
];
protected $casts = [
    'services' => 'array', // JSON decoded automatically
    'is_active' => 'boolean',
];
public function doctors(): HasMany {
    return $this->hasMany(Doctor::class);
}
public function appointments(): HasMany {
    return $this->hasMany(Appointment::class);
}
}

```

Appointment Model — Key Relationships & Scopes

```

<?php // app/Models/Appointment.php
class Appointment extends Model {
protected $casts = [
    'appointment_date' => 'date',
    'fee' => 'decimal:2',
    'auto_confirmed_at' => 'datetime',
    'checked_in_at' => 'datetime',
];
public function patient(): BelongsTo {
    return $this->belongsTo(Patient::class); }
public function doctor(): BelongsTo {
    return $this->belongsTo(Doctor::class); }
public function department(): BelongsTo {
    return $this->belongsTo(Department::class); }
public function diagnosis(): HasOne {
    return $this->hasOne(Diagnosis::class); }
// Query scope – confirmed appointments only
public function scopeConfirmed($query) {
    return $query->where('status', 'confirmed'); }
}

```

Doctor Model — Key Relationships

```

<?php // app/Models/Doctor.php
class Doctor extends Model {
    public function department(): BelongsTo {
        return $this->belongsTo(Department::class); }
    public function appointments(): HasMany {
        return $this->hasMany(Appointment::class); }
    public function diagnoses(): HasMany {
        return $this->hasMany(Diagnosis::class); }
    public function prescriptions(): HasMany {
        return $this->hasMany(Prescription::class); }
}

```

9.3 Controller Outline

```

<?php // app/Http/Controllers/Api/AppointmentController.php
class AppointmentController extends Controller {

    /** GET /api/appointments */
    public function index(Request $request): JsonResponse {
        $data = Appointment::with(['patient', 'doctor', 'department'])
        ->when($request->status, fn($q,$s) => $q->where('status',$s))
        ->when($request->doctor_id, fn($q,$id) => $q->where('doctor_id',$id))
        ->orderBy('appointment_date')->paginate(15);
        return response()->json($data);
    }

    /** POST /api/appointments */
    public function store(StoreAppointmentRequest $req): JsonResponse {
        $appt = Appointment::create($req->validated());
        return response()->json($appt, 201);
    }

    /** PATCH /api/appointments/{id}/confirm */
    public function confirm(Appointment $appointment): JsonResponse {
        $appointment->update([
            'status' => 'confirmed', 'auto_confirmed_at' => now(),
        ]);
        return response()->json(['message' => 'Confirmed.']);
    }
}

```

9.4 Route Definitions

```
// routes/api.php
use App\Http\Controllers\Api\{AppointmentController,
DepartmentController, DoctorController};

Route::middleware('auth:sanctum')->group(function () {
// Departments (read-only for non-admin)
Route::apiResource('departments', DepartmentController::class)
->only(['index', 'show']);

// Doctors
Route::apiResource('doctors', DoctorController::class)->only(['index', 'show']);
Route::get('departments/{dept}/doctors',
[DoctorController::class, 'byDepartment']);

// Appointments (full CRUD + custom actions)
Route::apiResource('appointments', AppointmentController::class);
Route::patch('appointments/{appointment}/confirm',
[AppointmentController::class, 'confirm']);
Route::patch('appointments/{appointment}/checkin',
[AppointmentController::class, 'checkIn']);

// Diagnostic results
Route::get('patients/{patient}/lab-results',
[LabResultController::class, 'forPatient']);
Route::get('patients/{patient}/scan-results',
[ScanResultController::class, 'forPatient']);
});
```

9.5 API Response Example

The JSON structure returned by GET /api/appointments/{id} with eager-loaded relationships:

```
{
  "id": 46,
  "appointment_date": "2026-05-04",
  "start_time": "09:00:00",
  "end_time": "09:30:00",
  "status": "confirmed",
  "treatment": "Cardiac Consultation",
  "fee": "300.00",
  "queue_number": 1,
  "patient": {
    "id": 13, "name": "Ahmed Hassan",
    "phone": "01553504020", "gender": "male"
  },
  "doctor": {
    "id": 7, "name": "Dr. Mohab Esam",
    "specialty": "Cardiology", "rating": 4.8
  },
  "department": {
    "id": 2, "name": "Cardiology", "slug": "cardiology"
  }
}
```

Note on Removed Content: The original document contained a section titled "Part 2: Analysis of the Inserted Data (Data Dumping)" which discussed specific appointment IDs, seeded sample records, Arabic character encoding tests, and fee statistics drawn from test data. This material has been excluded from the Database Design chapter as it constitutes test-phase validation analysis rather than a structural design artefact. It is recommended that this content be relocated to the Testing & Validation Chapter of the project, where it appropriately documents system behaviour under test conditions.